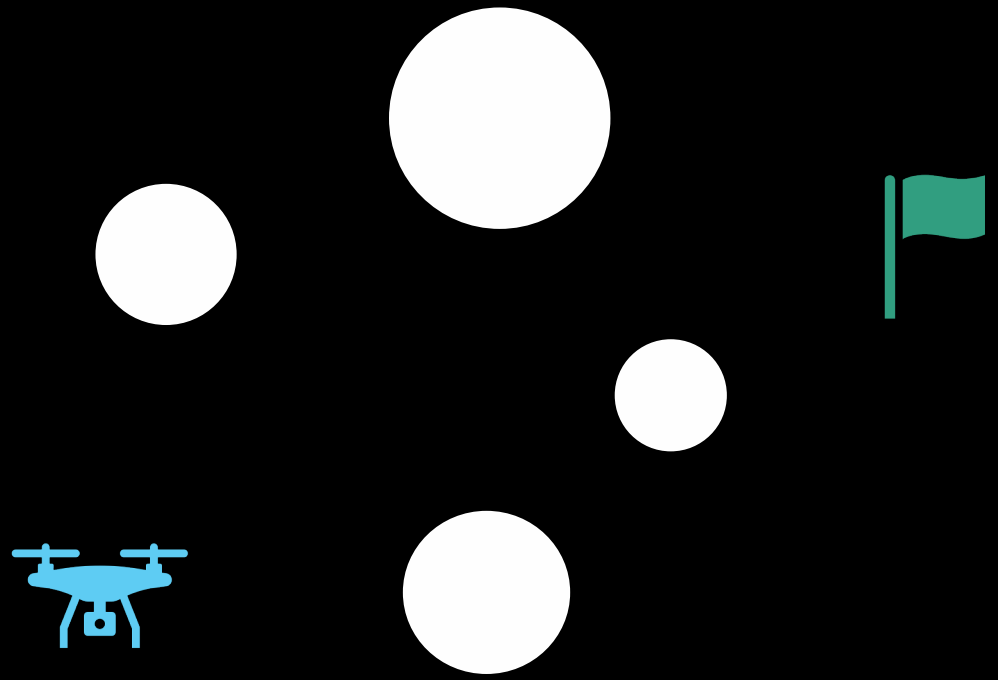
AA/EE/ME 548

Linear Multivariable Control

Optimization- and learning-based, open-loop & closed-loop control

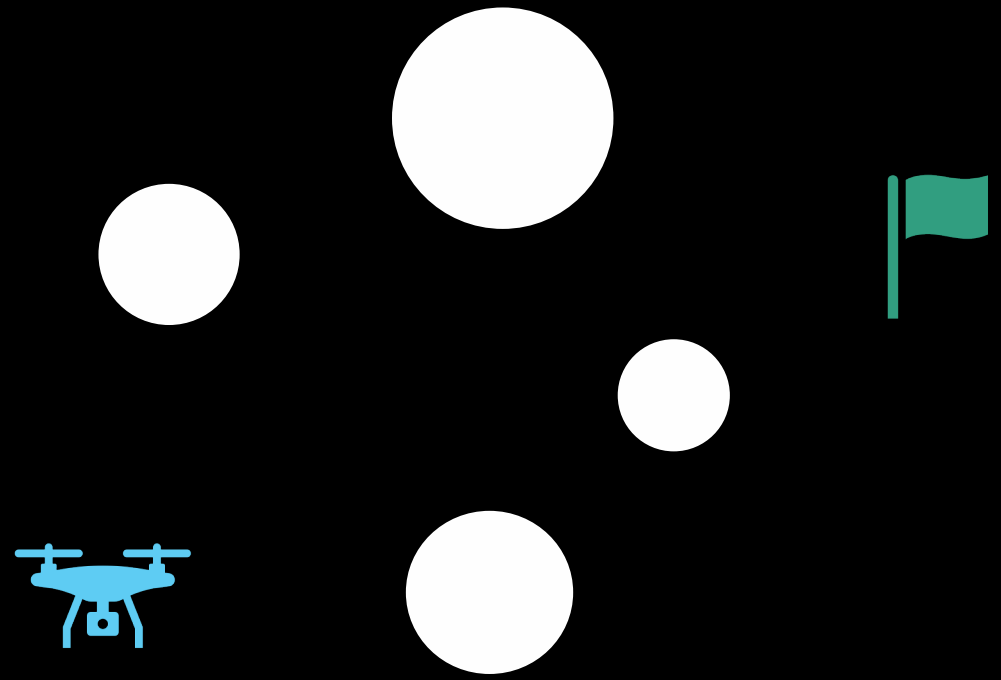
Spring 2026

How to compute a control to perform desired task (e.g., navigate to goal)?

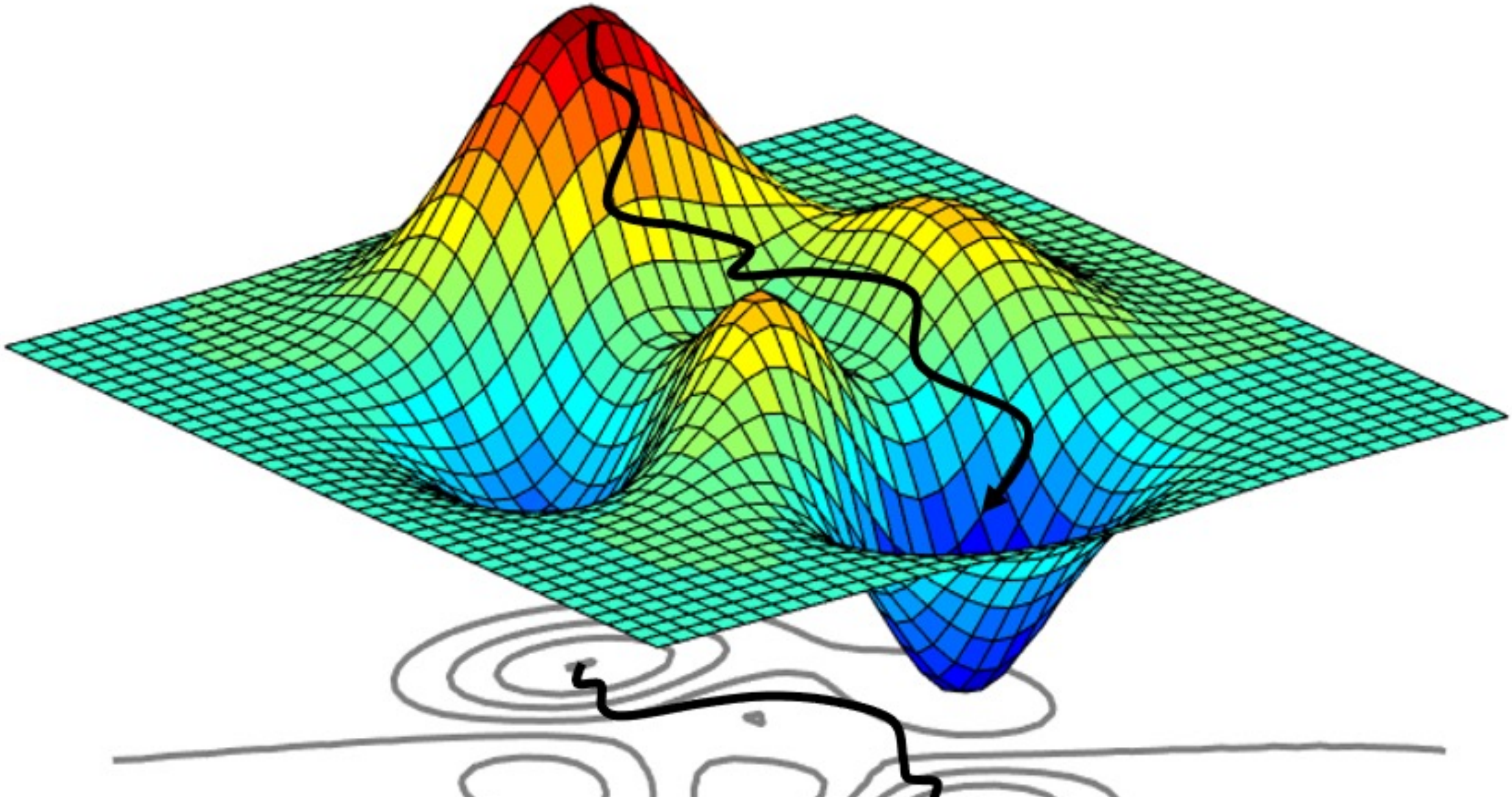


- Seek to know everything about the environment to make a *calculated* plan on what to do
- Use prior experience – apply *pattern matching* to obtain to decide on control
- Trial-and-error: try a bunch of things and see what works and doesn't work.

How to compute a control to perform desired task (e.g., navigate to goal)?



- Optimization-based control
- Imitation learning
- Reinforcement learning



Optimization-based control

Assumed knowledge

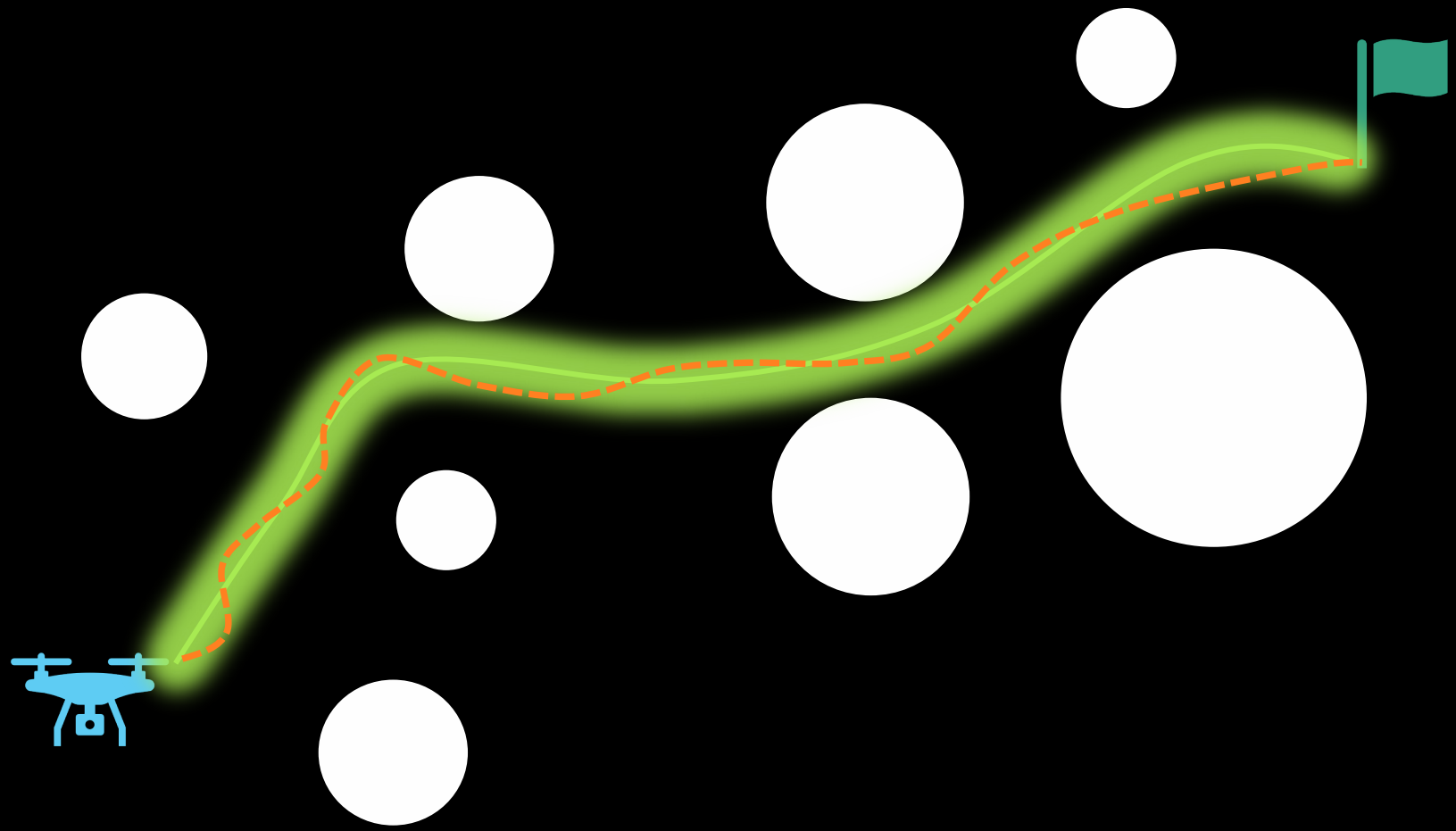
- System dynamics
  - $\dot{x} = f(x, u, d, t)$
- Environment information
  - Obstacle geometry and motion
- Task description
  - Objectives
  - Constraints

Optimization-based control: Pros and cons

Assumed knowledge

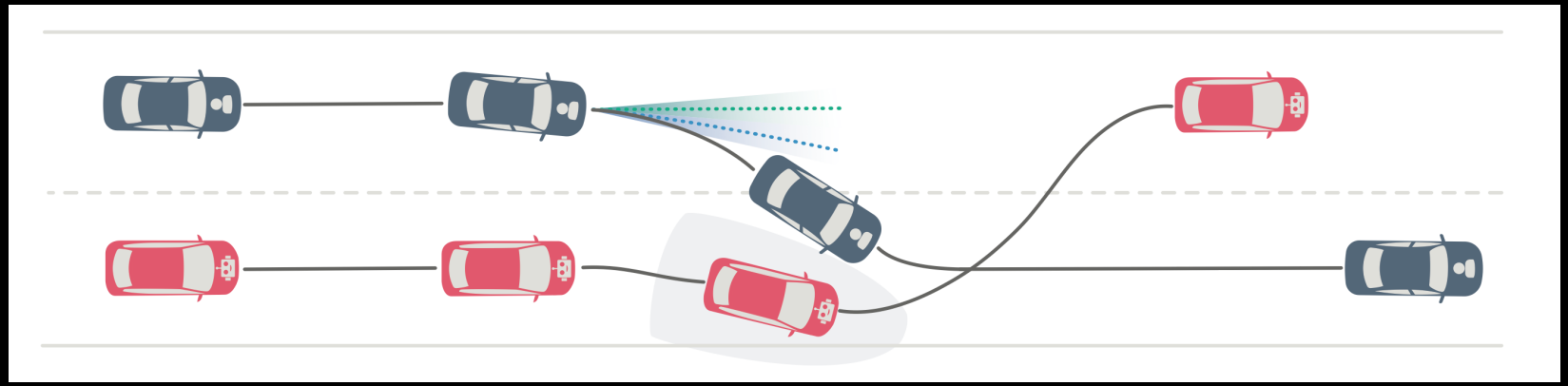
- System dynamics
  - $\dot{x} = f(x, u, d, t)$
- Environment information
  - Obstacle geometry and motion
- Task description
  - Objectives
  - Constraints

Open-loop for high-level plan, closed-loop for tracking



Human-robot interactions: Incorporating human prediction within robot planning

Robots interacting with humans is challenging due to the uncertainty in human behaviors.



Leung, K., Schmerling, E., Zhang, M., Chen, M., Talbot, J., Gerdes, J. C., and Pavone, M., On Infusing Reachability-Based Safety Assurance within Planning Frameworks for Human-Robot Vehicle Interactions, *Int. Journal of Robotics Research,* 2020

https://www.researchgate.net/publication/381197640_Interactive_and_Explainable_Robot_Learning_A_Comprehensive_Review

Imitation learning: Behavior cloning

Assumed knowledge

- Demonstrations of system performing the task

Idea

- Learn a policy that would reproduce the demonstration via *supervised learning*

Imitation learning: Generative modeling

Assumed knowledge

- Demonstrations of system performing the task

Idea

- Learn a policy that would reproduce the demonstrations by *matching the data distribution*

Imitation learning: Inverse optimal control

Assumed knowledge

- Demonstrations of system performing the task

Idea

- *Learn an objective function* that would reproduce the demonstrations

Imitation learning: Pros and cons

Assumed knowledge

- Demonstrations of system performing the task

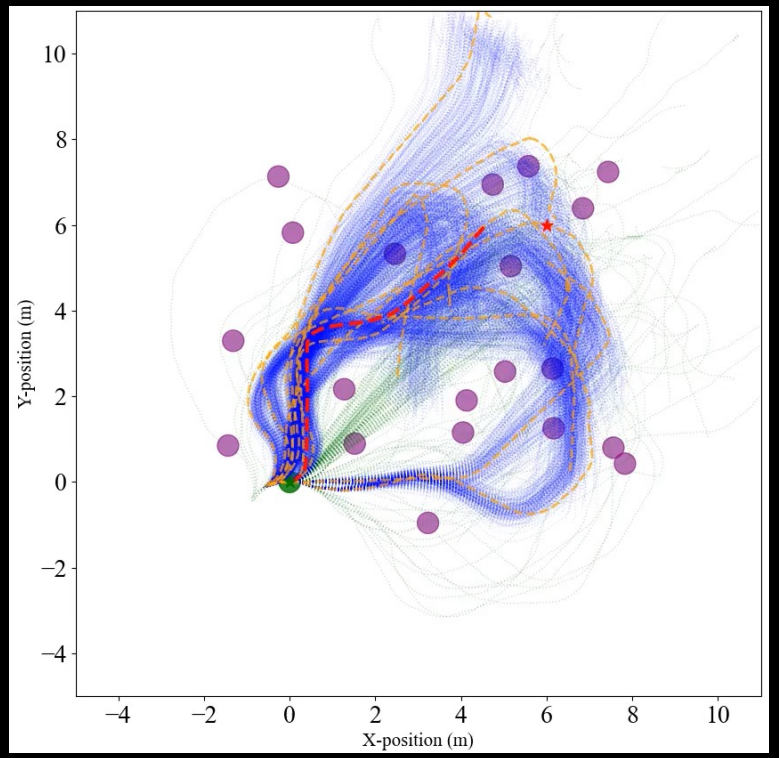
Idea

- Learn the policy that would reproduce the demonstrations

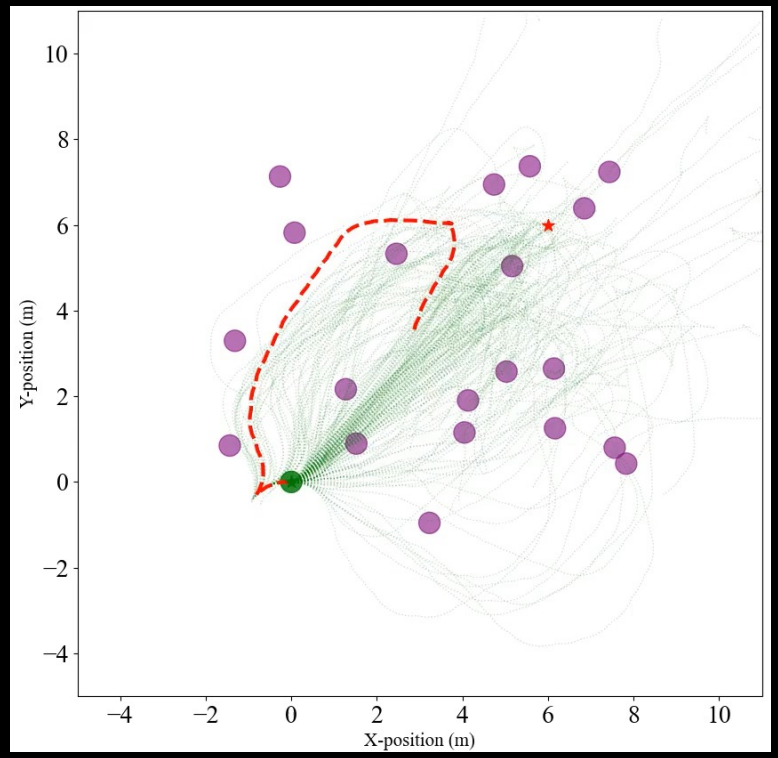

Physical Intelligence: https://www.pi.website/blog/olympics
autonomous, 4x, 00:00

Generation-refinement: Combining generative modeling with MPC

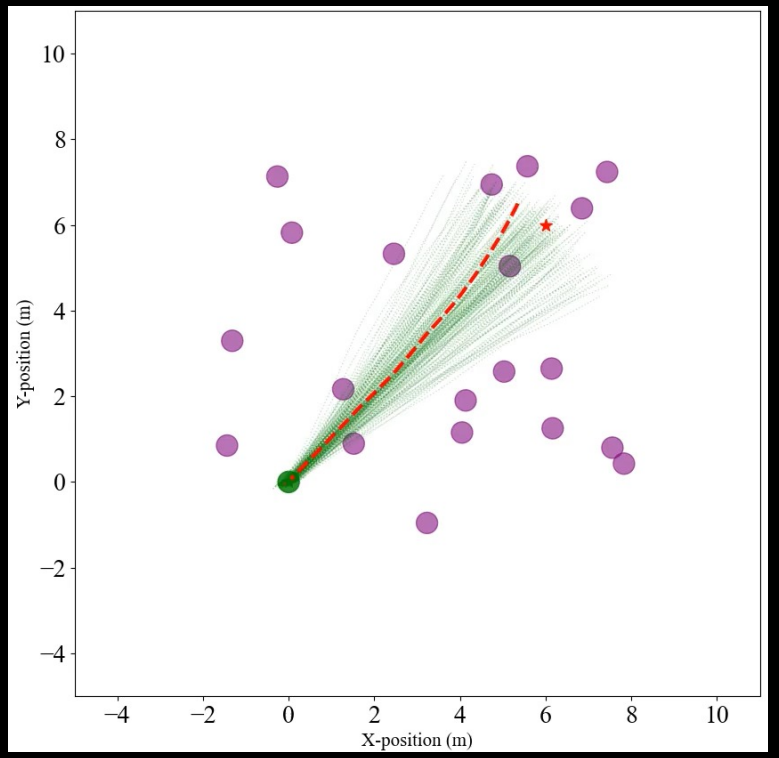
Generative model + optimization



Generative model only



Optimization only



Mizuta, K., and Leung, K., **Unified Generation-Refinement Planning: Bridging Guided Flow Matching and Sampling-Based MPC for Social Navigation**
In Proc. IEEE Conf. on Robotics and Automation, 2026 (accepted), https://arxiv.org/abs/2508.01192

Reinforcement learning

Assumed knowledge

- Interactions with the world (observation, transition, action)
- Reward signal

Reinforcement learning: Pros and cons

Assumed knowledge

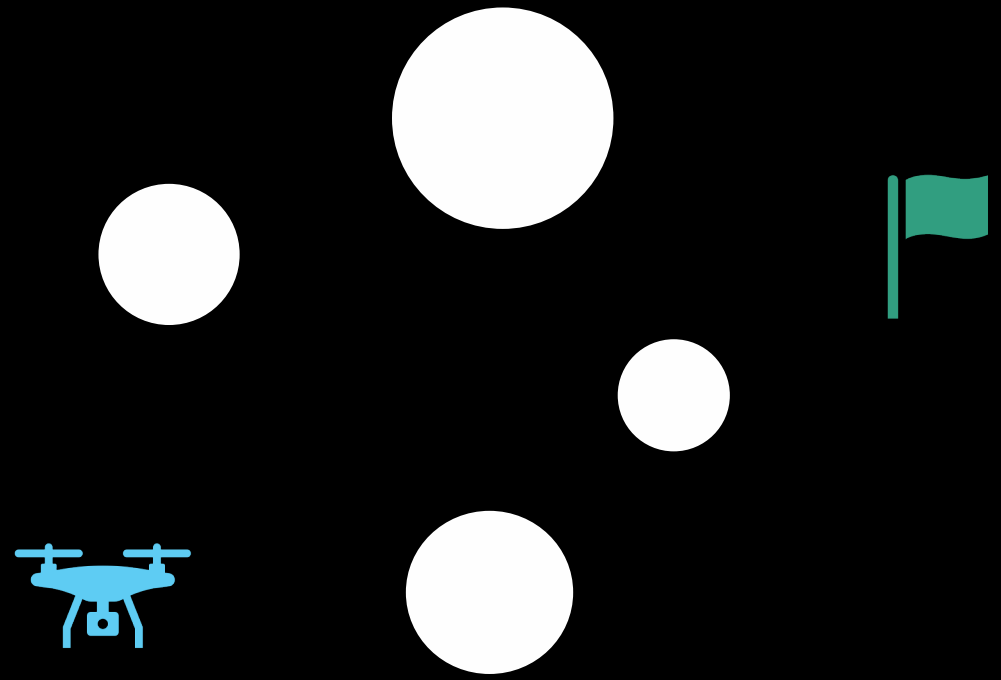
- Interactions with the world (observation, transition, action)
- Reward signal

https://eureka-research.github.io/

The new Atlas robot
from Boston Dynamics

https://www.youtube.com/shorts/DGlmZMTfFp8

How to compute a control to perform desired task (e.g., navigate to goal)?

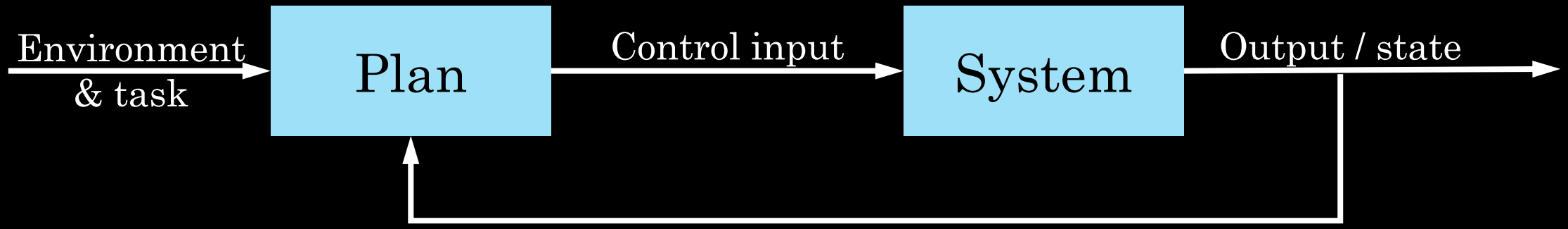


- Optimization-based control
- Imitation learning
- Reinforcement learning

Closed-loop vs open-loop control

Open-loop control: No feedback loop

Closed-loop control: No feedback loop



Open-loop control

Plan offline then execute with no updates online

E.g.,

Benefits

Limitations

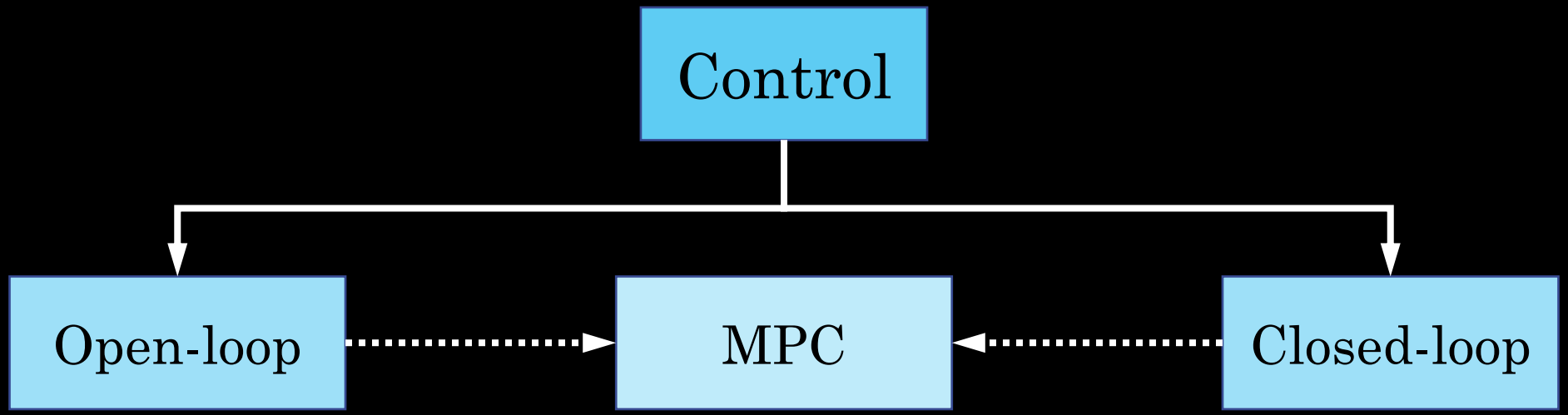
Closed-loop control

Execute controller $\pi(\cdot)$ online based on feedback on current output/state. Ideally, has insight into consequences of current actions

E.g.,

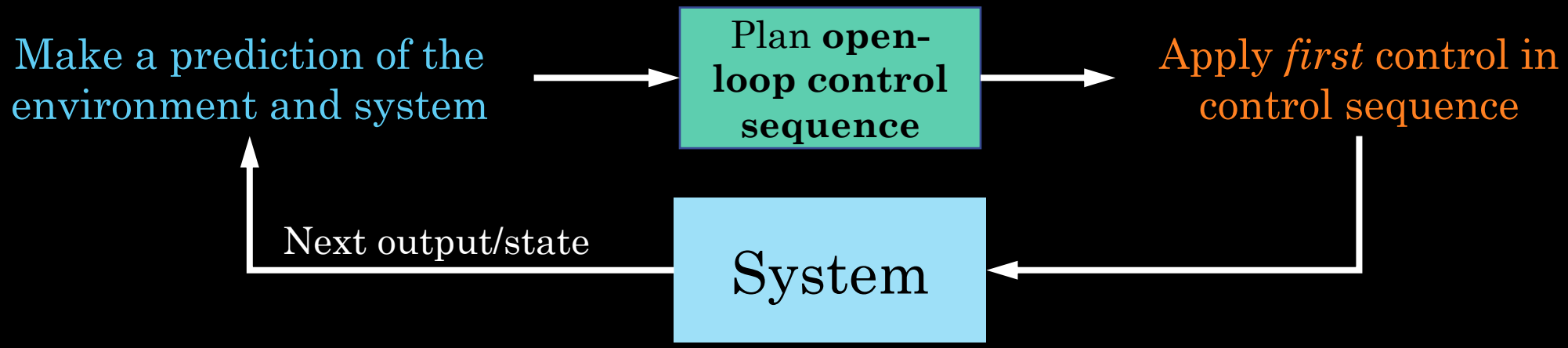
Benefits

Limitations


Control
Open-loop
MPC
Closed-loop

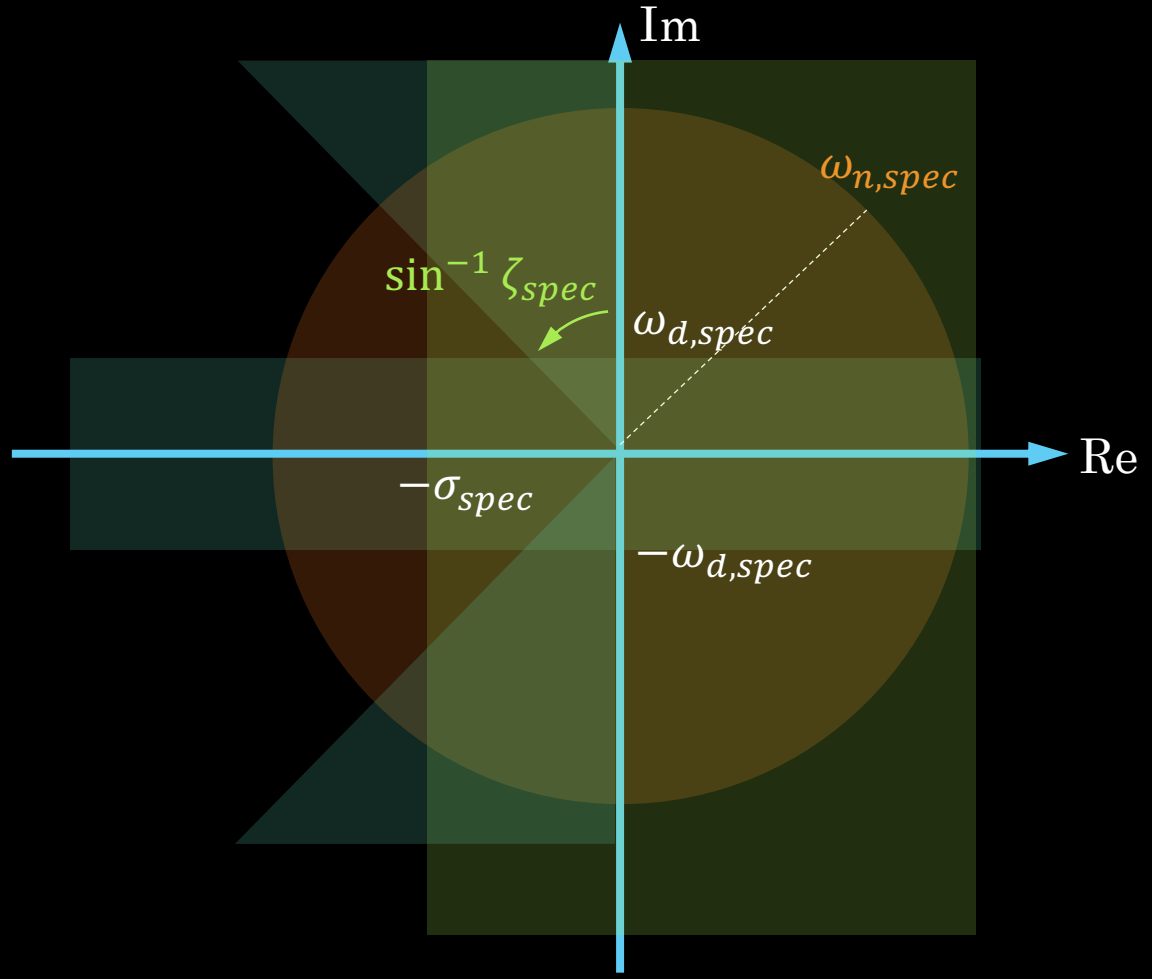
Model predictive control (MPC)
Closing the loop on open-loop control

Perform open-loop control repeatedly online using updated output/state and environment information.



Optimization-based control

Control design given specifications



Shaded areas mean not allowed

1% Settling time: $t_{1\%,s} = \frac{4.6}{\zeta\omega_n}$

Require: $t_s \leq t_{s,spec} \Rightarrow \sigma \geq \frac{4.6}{t_{s,spec}} = \sigma_{spec}$

Rise time: $t_r \approx \frac{1.8}{\omega_n}$

Require: $t_r \leq t_{r,spec} \Rightarrow \omega_n \geq \frac{1.8}{t_{r,spec}} = \omega_{n,spec}$

Peak time: $t_p = \frac{\pi}{\omega_n\sqrt{1-\zeta^2}}$

Require: $t_p \leq t_{p,spec} \Rightarrow \omega_d \geq \frac{\pi}{t_{p,spec}} = \omega_{d,spec}$

Maximum overshoot: $M_p = e^{\frac{-\pi\zeta}{\sqrt{1-\zeta^2}}}$

Require: $M_p \leq M_{p,spec} \Rightarrow \zeta^2 \geq \frac{(\ln M_{p,spec})^2}{\pi^2+(\ln M_{p,spec})^2} = \zeta_{spec}^2$

Specification- vs optimization-based control

Specification-based control

- Need controller to ensure a set of requirements are met
  - E.g., settling time, rise time, overshoot
- Unclear how to determine the "best" controller.

Optimization-based control

- Choose the *best* controller given a *cost function*
  - A is strictly better than B if $c(A) < c(B)$

Mathematical optimization

*The science of finding the **best** solution from a set of available alternatives*

$$\begin{aligned} \underset{x}{\text{minimize}} \quad & c(x) \\ \text{Type equation here.} \\ \text{subject to} \quad & g_i(x) \leq 0, \qquad i = 1, \dots, n_g \\ & h_j(x) = 0, \qquad j = 1, \dots, n_h \\ & x \in \mathcal{X} \end{aligned}$$

Mathematical optimization

Supervised learning

$$\min_{\theta} \quad \frac{1}{N} \sum_{(x,y)\in\mathcal{D}} \left(y - m_\theta(x)\right)^2$$

Mathematical optimization

Generative modeling

$$\min_{\theta} \quad \frac{1}{N} \sum_{x \in \mathcal{D}} -\log p_\theta(x)$$

Mathematical optimization

Resource allocation

$$
\begin{aligned}
\max_{x_1, x_2} \quad & p_1 x_1 + p_2 x_2 \\
\text{subject to} \quad & t_1 x_1 + t_2 x_2 \leq t_{\max} \\
& m_1 x_1 + m_2 x_2 \leq m_{\max} \\
& s_1 x_1 + s_2 x_2 \leq s_{\max}
\end{aligned}
$$

Types of optimization problems
Convex programs

- Linear program
- Quadratic program

Aside: Convex functions and convex sets

Types of optimization problems
Non-convex

- Mixed integer program
- Planning around obstacles

How to solve optimization problems

- Gradient descent
- Derivative-free methods
  - Sampling-based
- If convex, use beloved `cvxpy`
- Iterative methods: Make convex approximation, solve, then convexify about new solution

cvxpy

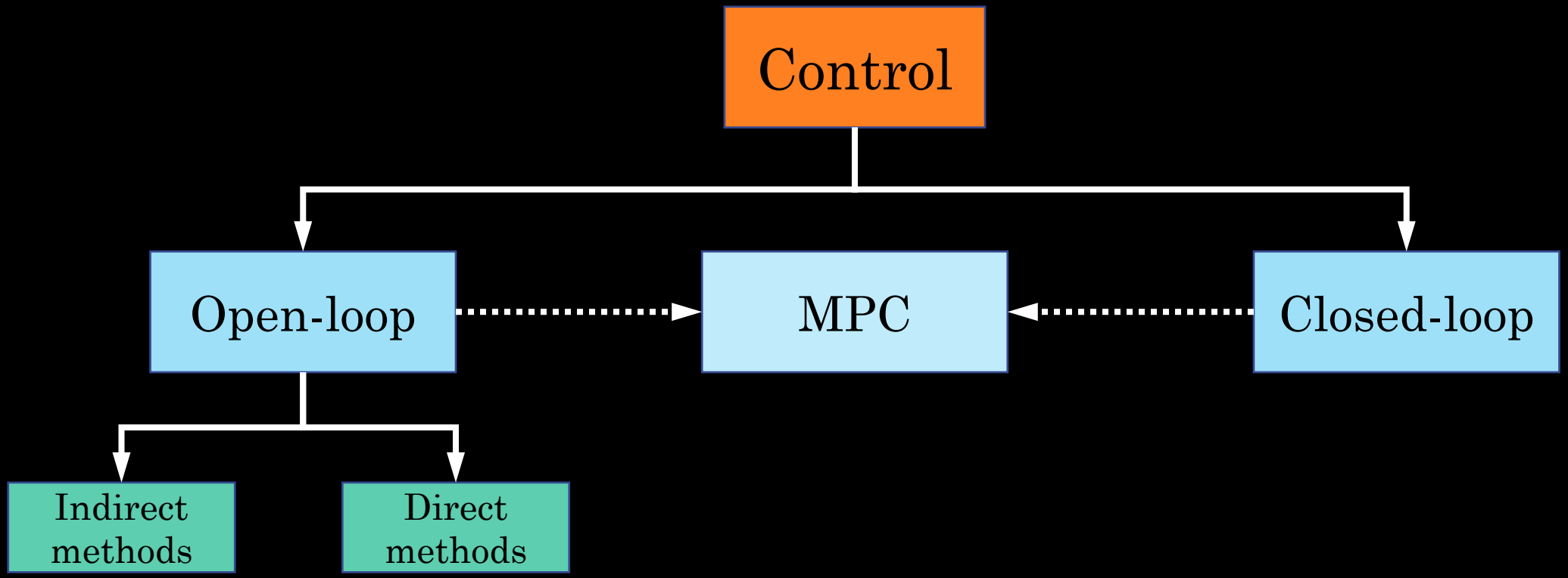
A domain-specific modeling language for (convex) mathematical optimization.

It lets you express the optimization problem in a natural way. User does not need to worry about the hairy details of the specific backend solver.

Trajectory optimization

Optimization-based control // Open-loop control

Additional reference: https://underactuated.mit.edu/trajopt.html


Control
Open-loop
MPC
Closed-loop
Indirect methods
Direct methods

Optimal control problem: Control synthesis as an optimization problem

$$
\begin{aligned}
(\mathrm{CT}) \qquad & \min_{u(\cdot)} && \int_0^T c(x(\tau), u(\tau), \tau)\, d\tau + c_T(x(T), T) \\
& \text{subject to} && \dot{x} = f(x, u, t) \\
& && g_i\big(x(\tau), u(\tau)\big) \leq 0, && i = 1, \ldots, n_g \\
& && h_j\big(x(\tau), u(\tau)\big) = 0, && j = 1, \ldots, n_h \\
& && x(0) = x_{\text{current}} \\
& && x \in \mathcal{X}, u \in \mathcal{U} \\
& && \tau \in [0, T]
\end{aligned}
$$

Approaches to solving the continuous time optimal control problem

Indirect method *("Optimize then discretize")*

Use calculus of variations to solve for optimality conditions then solve for the conditions.

(Results in a two-point boundary value problem)

Direct method *("Discretize then optimize")*

Discretize the problem to obtain finite-dimensional optimization problem.

(Results in solving a mathematical optimization program)

Indirect method: Pontryagin's Minimum Principle (PMP)

Hamiltonian of the system

$$H(x, u, \lambda, t) = c(x, u, t) + \lambda^T f(x, u, t)$$

λ is the *costate*, which represents sensitivity of your cost to changes in the state ($\lambda = \nabla V(x, t)$)

First-order necessary conditions for optimality

$$\frac{\partial H}{\partial \lambda} = \dot{x}, \qquad \frac{\partial H}{\partial x} = -\dot{\lambda}, \qquad \frac{\partial H}{\partial u} = 0$$

Results in 2PBVP. Problem becomes more challenging with controls & state constraints.

Direct methods: Turn into finite-dimensional problem

Single Shooting

$$\min_{u_0,\ldots,u_{N-1}} \quad \sum_{k=0}^{N-1} c(x_k, u_k, t_k) \quad + \quad c_N(x_N, t_N)$$

$$\begin{aligned}
\text{subject to} \quad & x_{k+1} = f(x_k, u_k, t_k) && k = 0, \ldots, N-1 \\
& g_i(x_k, u_k) \leq 0, && i = 1, \ldots, n_g \\
& h_j(x_k, u_k) = 0, && j = 1, \ldots, n_h \\
& x_0 = x_{\text{current}} \\
& x_k \in \mathcal{X}, u_k \in \mathcal{U}, && k = 0, \ldots, N
\end{aligned}$$

Multiple shooting

$$\min_{u_0,\ldots,u_{N-1},x_0,\ldots,x_N} \quad \sum_{k=0}^{N-1} c(x_k, u_k, t_k) \quad + \quad c_N(x_N, t_N)$$

$$\begin{aligned}
\text{subject to} \quad & x_{k+1} = f(x_k, u_k, t_k) && k = 0, \ldots, N-1 \\
& g_i(x_k, u_k) \leq 0, && i = 1, \ldots, n_g \\
& h_j(x_k, u_k) = 0, && j = 1, \ldots, n_h \\
& x_0 = x_{\text{current}} \\
& x_k \in \mathcal{X}, u_k \in \mathcal{U}, && k = 0, \ldots, N
\end{aligned}$$

(Multiple) Shooting

Collocation method

Introduce **algebraic constraints** to enforce the system dynamics at those nodes.

e.g., Trapezoidal collocation

$$x_{k+1} = x_k + \frac{\Delta t}{2}\left(f(x_k, u_k) + f(x_{k+1}, u_{k+1})\right)$$

Similar thing for integral in objective

Psuedospectral

Represent control and states as *polynomials* and evaluate at quadrature nodes

$$x(t) = \sum_{j=0}^{N} c_j p_j(t), \qquad u(t) = \sum_{j=0}^{N} d_j p_j(t)$$

- Use “special” polynomial basis and nodes for better numerical stability and properties
  - Chebyshev, Lagrange, Legendre

In all these methods, often results in *nonlinear program*

- The dynamics are nonlinear
- Objective is nonlinear
- Constraints are nonlinear

$$\min_{u_0,\ldots,u_{N-1},x_0,\ldots,x_N} \quad \sum_{k=0}^{N-1} c(x_k, u_k, t_k) \quad + \quad c_N(x_N, t_N)$$

$$\begin{aligned}
\text{subject to} \quad & x_{k+1} = f(x_k, u_k, t_k) && k = 0, \ldots, N-1 \\
& g_i(x_k, u_k) \leq 0, && i = 1, \ldots, n_g \\
& h_j(x_k, u_k) = 0, && j = 1, \ldots, n_h \\
& x_0 = x_{\text{current}} \\
& x_k \in \mathcal{X}, u_k \in \mathcal{U}, && k = 0, \ldots, N
\end{aligned}$$

NLP solvers

- Use a nonlinear program solver
- Matlab `fmincon`

Description

Nonlinear programming solver.

Finds the minimum of a problem specified by

$$\min_x f(x) \text{ such that } \begin{cases} c(x) \le 0 \\ ceq(x) = 0 \\ A \cdot x \le b \\ Aeq \cdot x = beq \\ lb \le x \le ub, \end{cases}$$

NLP solvers

- Use a nonlinear program solver
- Matlab `fmincon`
- `scipy.optimize.minimize`

scipy.optimize.

minimize

```
minimize(fun, x0, args=(), method=None, jac=None, hess=None, hessp=None,
bounds=None, constraints=(), tol=None, callback=None, options=None)
```

Minimization of scalar function of one or more variables. [source]

NLP solvers

- Use a nonlinear program solver
- Matlab `fmincon`
- `scipy.optimize.minimize`
- Many others, commercial solvers, free academic licenses.
- Lots of open-source ones too

But what is happening under the hood in these NLP solvers?

- Really smart with linear algebra and other tricks
  - E.g., exploiting sparsity, well-conditioning the problem
- Some sort of **log-barrier method**, turning constrained to unconstrained
- Apply some form of approximate method
- Smarter gradient descent
  - Use second order information, smarter line search

Sequential Quadratic Program (SQP)

$$\min_{u_0,\ldots,u_{N-1},x_0,\ldots,x_N} \quad \sum_{k=0}^{N-1} c(x_k, u_k, t_k) \quad + \quad c_N(x_N, t_N)$$

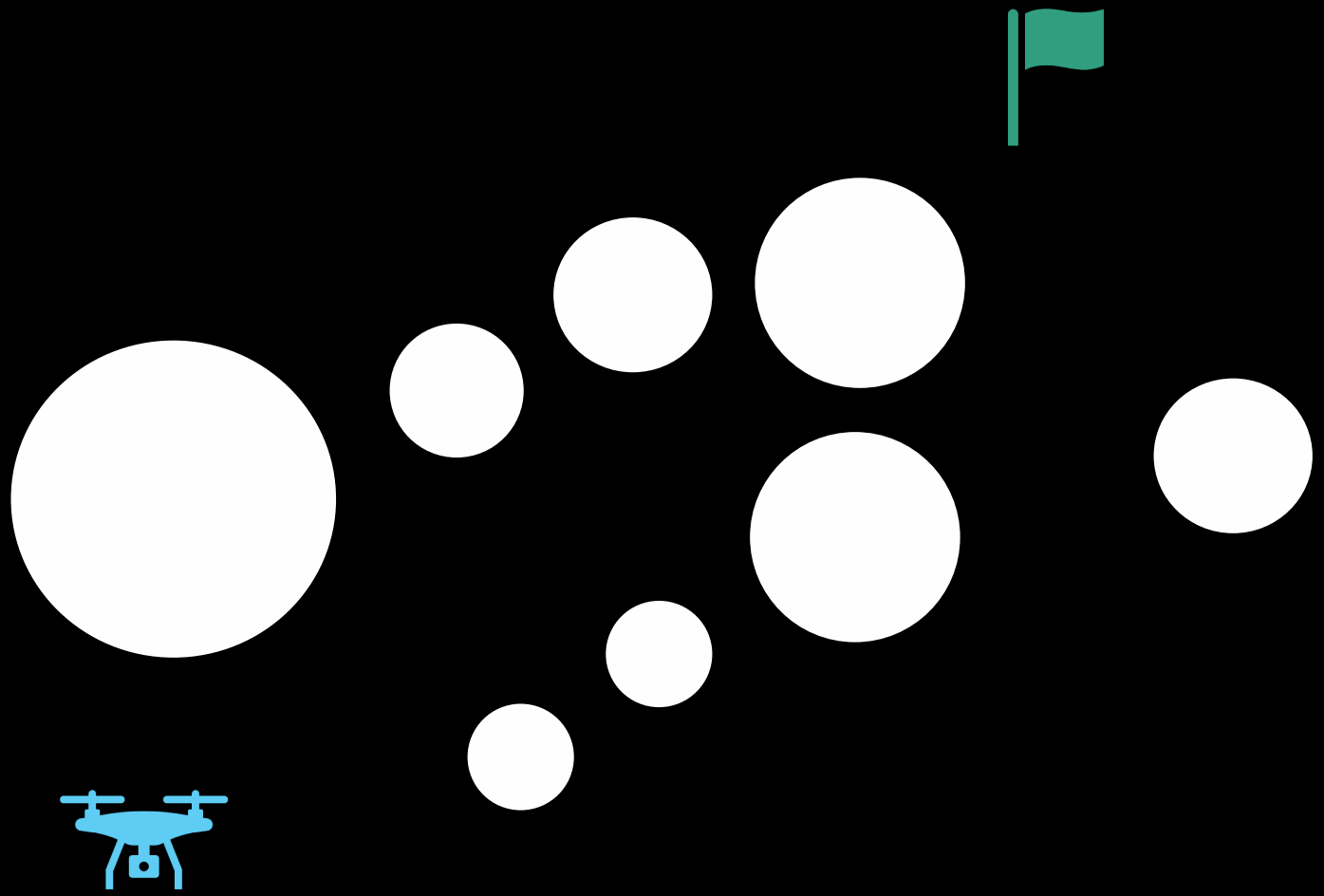
$$\begin{aligned}
\text{subject to} \quad & x_{k+1} = f(x_k, u_k, t_k) && k = 0, \ldots, N-1 \\
& g_i(x_k, u_k) \leq 0, && i = 1, \ldots, n_g \\
& h_j(x_k, u_k) = 0, && j = 1, \ldots, n_h \\
& x_0 = x_{\text{current}} \\
& x_k \in \mathcal{X}, u_k \in \mathcal{U}, && k = 0, \ldots, N
\end{aligned}$$

Sequential quadratic programming

- Approximate the problem as a quadratic program and repeatedly solve

1. Start with initial guess $(\tilde{x}_0, \tilde{x}_1, \ldots, \tilde{x}_{T+1}, \tilde{u}_0, \tilde{u}_1, \ldots, \tilde{u}_T)$
2. Linearize dynamics about guess $x_{t+1} \approx A_t x_t + B_t u_t + C_t$
3. Linearize constraints $g(x, u) \leq 0$: $G_t x_t + H_t u_t + h_t \leq 0$
4. Quadratize cost $c(x_t, u_t) \approx \frac{1}{2} x_t^T Q_t x_t + \frac{1}{2} u_t R_t u_t + p_t^T x_t + r_t^T u_t$
5. Add trust region penalty to cost: $x_t \approx \tilde{x}_t, u_t \approx \tilde{u}_t$
6. Solve QP! Solution = new guess
7. Go to step 1.

Initialization matters



https://ieeexplore.ieee.org/abstract/document/11068406

What if the problem is *really* hard?

Or when there's lots of dynamic obstacles

What are the challenge that arise?

- Cost, dynamics, constraints are *ugly* to work with
  - Difficult to differentiate
  - Maybe blackbox
  - Maybe learned (neural network)
- Stochastic elements
  - Uncertainty in the environment.
  - Require drawing many samples
- Generally, the standard NLP / gradient descent framework is not applicable.

Sampling-based: Model Predictive Path Integral (MPPI)

Information Theoretic MPC for Model-Based Reinforcement Learning

Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews,
James M. Rehg, Byron Boots, and Evangelos A. Theodorou

***Abstract*— We introduce an information theoretic model predictive control (MPC) algorithm capable of handling complex cost criteria and general nonlinear dynamics. The generality of the approach makes it possible to use multi-layer neural networks as dynamics models, which we incorporate into our MPC algorithm in order to solve model-based reinforcement learning tasks. We test the algorithm in simulation on a cart-pole swing up and quadrotor navigation task, as well as on actual hardware in an aggressive driving task. Empirical results demonstrate that the algorithm is capable of achieving a high level of performance and does so only utilizing data collected from the system.**

Fig. 1. Aggressive driving with MPPI and neural network dynamics.

https://www.overland.ai/
https://homes.cs.washington.edu/~bboots/files/InformationTheoreticMPC.pdf

Sampling-based: Model Predictive Path Integral

1. Start with nominal trajectory $\tilde{\boldsymbol{u}}$
2. Add noise to it to generate many trajectories $\boldsymbol{\delta}_k$
3. Evaluate cost of each trajectory c_k
4. Compute weight for each trajectory $w_k = \frac{e^{-c_k}}{\sum_j e^{-c_j}}$
5. Compute weighted sum over controls to compute control $\boldsymbol{u}^* = \tilde{\boldsymbol{u}} + \sum_k w_k \boldsymbol{\delta}_k$

Idea: Take a noisy estimate of the gradient and move along that direction

MPPI

Generating good warm starts for optimization

https://ieeexplore.ieee.org/abstract/document/11068406